

Problem Set 3

A. The Problem Set

←

✓

Difficulty: NA

🌙

Expand

🔖

Statement

Submissions

Solution

Help

🔔

Learn the building blocks of Python

Take our programming courses to learn the Python language

Start Learning

The Problem Set

As we all know that team LaZYProblemSetters is gearing up for ICPC 2015 and have finally reached in front of a computer together to solve a problem set. Since they have already wasted too much time procrastinating, they have decided to manage the problem set with more caution. They are now ordering the problems according to their difficulty levels instead of solving all the easy problems as they used to do earlier.

For this practice round, everybody has pitched in a few problems of their choice. So in the end, they have a set of N problems ordered as $P_1, \dots, P_N$, where P_i denotes the i^{th} problem. For each problem that was brought in by any team member, he assigned an integer D_i which according to him is the difficulty level of that problem.

The difficulty level assigned to a problem is exactly the number of time units one expects to solve that particular problem. However, as we all know, that assigning difficulty to a problem is really subjective and the assigned difficulty level of the problem may change when all the three team members look at that problem. Since, they are busy solving the problem, they have asked you to help in giving them the information they need about the problem set.

Following is the list of all of their requirements, for which they need your help:

1. Update the difficulty level of a problem.

2. Tell them the difficulty level of the most difficult problem in the problem set.

3. The total time it would take them to solve a set of problems.

4. Whether a list of consecutive problems are in some order of difficulty.

◦ P_x to P_y are in increasing order of difficulty if $D_x \leq D_{x+1} \leq \dots \leq D_y$

◦ P_x to P_y are in decreasing order of difficulty if $D_x \geq D_{x+1} \geq \dots \geq D_y$

To make it easier, faster and accurate, they have decided to communicate with you as follows:

1. **U x d**
The difficulty level of the problem P_x has been updated to d . Hence $D_x = d$.

2. **M x y**
What is the difficulty level of the most difficult problem in range $P_x, \dots, P_y$.

3. **S x y**
How much total time it would take them to solve all the problems in range $P_x, \dots, P_y$.

4. **I x y**
Whether the problems in range $P_x, \dots, P_y$ are in increasing order of difficulty. Print 1 if they are, 0 otherwise.

5. **D x y**
Whether the problems in range $P_x, \dots, P_y$ are in decreasing order of difficulty. Print 1 if they are, 0 otherwise.

C++

```
54
55
56     if (type == 'U') {
57         update(x - 1, y, root_index, range_tree);
58         // cout << y << '\n';
59     } else if (type == 'M') {
60         cout << search(x - 1, y - 1, root_index, range_tree).max << '\n';
61     } else if (type == 'S') {
62         cout << search(x - 1, y - 1, root_index, range_tree).sum << '\n';
63     } else if (type == 'I') {
64         cout << search(x - 1, y - 1, root_index, range_tree).is_inc << '\n';
65     } else if (type == 'D') {
66         cout << search(x - 1, y - 1, root_index, range_tree).is_dec << '\n';
67     }
68 }
69
70
71 int build(int start, int end, vector<int>& problems, vector<Node>& range_tree) {
72     if (start == end) {
73         Node n;
74         n.max = problems[start];
75         n.sum = problems[start];
76         n.is_inc = 1;
77         n.is_dec = 1;
78         range_tree.push_back(n);
79         return start;
80     }
81     int mid = (start + end) / 2;
82     int left = build(start, mid, problems, range_tree);
83     int right = build(mid + 1, end, problems, range_tree);
84     Node n;
85     n.max = max(range_tree[left].max, range_tree[right].max);
86     n.sum = range_tree[left].sum + range_tree[right].sum;
87     n.is_inc = range_tree[left].is_inc && range_tree[right].is_inc;
88     n.is_dec = range_tree[left].is_dec && range_tree[right].is_dec;
89     range_tree.push_back(n);
90     return mid;
91 }
```

Test against Custom Input

Your program will run with no input

✓

Correct Answer

Submission ID: 1290107053

Time:

1.15s

Sub-Task	Task #	Result (time)
1	0	Correct (0.04)
1	1	Correct (0.53)
1	2	Correct (1.04)
1	3	Correct (1.07)
1	4	Correct (1.15)
Subtask Score: 100%		Result - Correct
		Total Score = 100%

🔗 Visualize Code

▶ Run

Submit

Process

The question provides n problems with D_i difficulty level. There are 4 operations:

- Update the difficulty level of a problem.
- Tell the difficulty level of the most difficult problem in range P_x to P_y .
- Tell the total time it would take them to solve all the problems in range P_x to P_y .
- Tell whether the problems in range $P_x, \dots, P_y$ are in increasing order of difficulty.
- Tell whether the problems in range $P_x, \dots, P_y$ are in decreasing order of difficulty.

Challenges and Overcoming

Clearly, since this is a range problem and max, sum, increasing and decreasing are binary associative operators, we can use range tree in this case.

Then, I need 3 methods to handle the range tree.

1. `int build(int start, int end, vector<int>& problems, vector<Node>& range_tree)`: this function takes all the problems in a vector, and the range we want to build the range tree. Initially, the range should be that whole problem set. This method can recursively build the range tree by having left subtree and right subtree built first and combine this 2 nodes to build the current node.
2. `void update(int x, int new_value, int root_index, vector<Node>& range_tree)`: this function can update the tree due to the update of the difficulty of a certain question. It will recursively update the tree by updating the leaf node first and update the side effect among the path from that leaf node to the root node.
3. `QueryResult search(int start, int end, int root_index, vector<Node>& range_tree)`: given the range, this method search for the QueryResult in that range.

The sum operation and max operation are standard ways of using range tree. Checking if the difficulties are sorted require extra information in `Node`. I have `bool is_inc`, `bool is_dec`, `int first_val`, `int last_val` in the `struct Node`. Everytime when I need to check if the current range is asc/desc, I will break the range into 2 part, and if these 2 subrange are both asc and last_val of first range is less or equal to the first_val of the second range, then the whole range is asc. This is similar to desc case.